توجه به این نکته مهم است که فراخوانی pthread_cond_signal() قفل انحصار متقابل را رها نمی‌کند. فراخوانی بعدی به pthread_mutex_unlock() است که قفل انحصار متقابل را رها می‌سازد. به‌محض این که قفل انحصار متقابل رها می‌شود، نخ سیگنال دیده مالک قفل انحصار متقابل می‌شود و کنترل را از فراخوانی pthread_cond_wait() بر می‌گرداند.

ما چندین مسأله و پروژه‌ی برنامه‌نویسی در انتهای همین فصل تدارک دیده‌ایم که از قفل‌های انحصار متقابل نخ‌های پوزیکس و متغیرهای شرط، در کنار سمافورهای پوزیکس استفاده می‌کنند.

۷-۴ همگامی در جاوا

زبان جاوا و API آن از ابتدای این زبان، یک پشتیبانی غنی برای همگامی نخ فراهم ساخته‌اند. ما در این بخش، در ابتدا مانیتورهای جاوا، راهکار اصلی همگامی جاوا را پوشش می‌دهیم. سپس، سه راهکار دیگری را مطرح می‌کنیم که در نسخه‌ی 1.5 معرفی شدند: قفل‌های چندمدخلی[1]، سمافورها، و متغیرهای شرط. ما این موارد را آوردیم چون آن‌ها معرف رایج‌ترین راهکارهای قفل‌گذاری و همگامی هستند. با این وجود، API جاوا امکانات زیادی فراهم می‌کند که ما در این کتاب پوشش نمی‌دهیم—برای مثال، پشتیبانی از متغیرهای یکپارچه و دستورالعمل CAS.

۷-۴-۱ مانیتورهای جاوا

جاوا یک راهکار همروندی مشابه یک مانیتور را برای همگامی نخ فراهم می‌کند. ما این راهکار را با کلاس BoundedBuffer (شکل ۷-۹) نمایش می‌دهیم که یک راه‌حلی را برای مسئله‌ی بافر محدود پیاده‌سازی می‌کند که در آن، تولیدکننده و مصرف‌کننده به‌ترتیب متدهای insert() و remove() را احضار می‌نمایند.

```
public class BoundedBuffer<E>
{
        private static final int BUFFER_SIZE = 5;
        private int count, in, out;
        private E[] buffer;
        public BoundedBuffer() {
                count = 0;
                in = 0;
                out = 0;
                buffer = (E[]) new Object[BUFFER_SIZE];
        }
        /* Producers call this method */
        public synchronized void insert(E item) {
                /* See Figure 7.11 */
        }
```

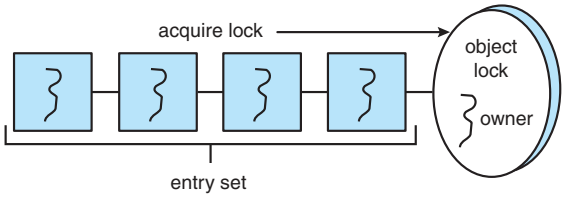
1. Reentrant locks

```
        /* Consumers call this method */
        public synchronized E remove() {
                /* See Figure 7.11 */
        }
}
```

شکل ۷–۹ بافر محدود با استفاده از همگامی جاوا.

هر شئ در جاوا، یک قفل منفرد مرتبط دارد. زمانی که متدی synchronized اعلام می‌شود، فراخوانی متد مستلزم کسب مالکیت قفل شئ است. ما متد همگام را با گذاشتن کلمه‌ی کلیدی synchronized در تعریف متد اعلان می‌کنیم، همچون متدهای ()insert و ()remove در کلاس BoundedBuffer.

احضار یک متد synchronized مستلزم مالکیت قفل بر روی یک نمونه‌ی شیئی از BoundedBuffer می‌باشد. اگر مالکیت قفل پیش از این در اختیار نخ دیگری باشد، نخی که متد synchronized را فراخوانی کرده است، مسدود می‌گردد و در **مجموعه‌ی ورودی** قفل شئ قرار داده می‌شود. مجموعه‌ی ورودی، مجموعه‌ای از نخ‌هاست که در انتظار در دسترس قرارگرفتن قفل به سر می‌برند. اگر قفل در زمان فراخوانی متد synchronized در دسترس باشد، نخی که متد را فراخوانی کرده است مالک قفل شئ می‌گردد و می‌تواند به متد وارد شود. زمانی که نخ از متد خارج می‌شود قفل رها می‌گردد. اگر موقعی که قفل رها می‌شود، مجموعه‌ی ورودی قفل خالی نباشد، JVM به‌دلخواه یک نخ را از این مجموعه انتخاب می‌کند تا مالک قفل شود. (موقعی که می‌گوییم «به‌دلخواه»، منظور ما این است که توصیف الزامی نمی‌گذارد که نخ‌های این مجموعه به یک ترتیب خاصی سازمان‌دهی شوند. با این وجود، اکثر ماشین‌های مجازی، نخ‌های مجموعه‌ی ورودی را در عمل مطابق یک سیاست FIFO مرتب می‌کنند.) شکل ۷–۱۰، چگونگی عملیات مجموعه‌ی ورودی را نمایش می‌دهد.



شکل ۷–۱۰ مجموعه‌ی ورودی یک قفل.

هر شئ علاوه بر داشتن یک قفل، یک **مجموعه‌ی منتظر** مرتبط شامل یک مجموعه نخ نیز خواهد داشت. این مجموعه‌ی منتظر در آغاز تهی است. موقعی که یک نخ وارد یک متد synchronized می‌شود، مالک قفل شئ می‌گردد. با این وجود، این نخ ممکن است تعیین کند که قادر به ادامه نیست چون یک شرط خاصی برآورده نشده است. برای مثال، این شرایط، اگر تولیدکننده متد ()insert را فراخوانی کند و بافر پر باشد اتفاق خواهد افتاد. در ادامه، نخ قفل را آزاد کرده و منتظر خواهد ماند تا این که شرطی که به آن اجازه‌ی ادامه‌ی کار می‌دهد برآورده شود.

موقعی که یک نخ متد ()wait را فراخوانی می‌کند، سناریوی زیر اتفاق می‌افتد:

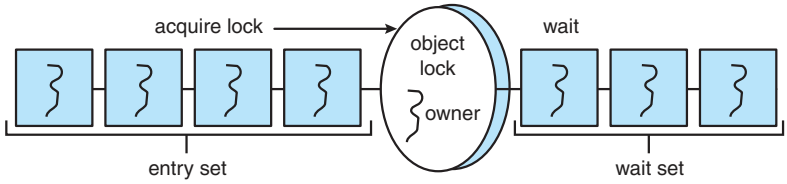
۱- نخ، قفل شئ را رها می‌کند.
۲- حالت نخ، مسدود انتخاب می‌شود.
۳- نخ در مجموعه‌ی منتظر شئ گذاشته می‌شود.

مثال شکل ۷-۱۱ را در نظر بگیرید. اگر تولیدکننده متد ()insert را فراخوانی کند و ببیند که بافر پر است، متد ()wait را فراخوانی می‌کند. این فراخوانی قفل را رها می‌سازد، تولیدکننده را مسدود کرده، و تولیدکننده را در مجموعه‌ی منتظر شئ قرار می‌دهد. چون تولیدکننده قفل را رها کرده است، مصرف‌کننده نهایتاً وارد متد ()remove می‌شود، که در آنجا یک جایی در بافر برای تولیدکننده آزاد می‌کند. شکل ۷-۱۲، مجموعه‌های ورودی و منتظر یک قفل را نمایش می‌دهد. (توجه کنید هرچند که ()wait می‌تواند استثنای InterruptedException را پرتاب کند، ما به‌خاطر خوانایی و سادگی کد از آن صرف‌نظر می‌کنیم.)

```
/* Producers call this method */
public synchronized void insert(E item) {
        while (count == BUFFER_SIZE) {
                try {
                        wait();
                }
                catch (InterruptedException ie) { }
        }
        buffer[in] = item;
        in = (in + 1) % BUFFER_SIZE;
        count++;
        notify();
}

/* Consumers call this method */
public synchronized E remove() {
        E item;
        while (count == 0) {
                try {
                        wait();
                }
                catch (InterruptedException ie) { }
        }
        item = buffer[out];
        out = (out + 1) % BUFFER_SIZE;
        count--;
        notify();
        return item;
}
```

شکل ۷-۱۱ متدهای ()insert و ()remove با استفاده از ()wait و ()notify.



شکل ۷-۱۲ مجموعه‌های ورودی و انتظار.

چگونه نخ مصرف‌کننده به تولیدکننده علامت می‌دهد که اکنون می‌تواند پیشروی داشته باشد؟ طبق معمول، موقعی که نخی از یک متد synchronized خارج می‌شود، نخ خارج شده فقط قفل مرتبط با شئ را رها می‌کند که احتمالاً با این کار، نخی از مجموعه‌ی ورودی برداشته شده و مالکیت قفل را در اختیار می‌گیرد. با این وجود، در انتهای متدهای ()insert و ()remove، یک فراخوانی به متد ()notify داریم. فراخوانی ()notify:

۱- نخ دلخواه T را از لیست نخ‌های مجموعه‌ی منتظر بر می‌دارد
۲- T را از مجموعه‌ی منتظر به مجموعه‌ی ورودی جابجا می‌کند
۳- حالت T را از مسدود به اجرایی تغییر می‌دهد

اکنون T به‌منظور رقابت برای قفل با سایر نخ‌ها آمادگی دارد. همین که T دوباره کنترل قفل را کسب می‌کند، از ()wait که فراخوانی کرده بر می‌گردد، جایی که ممکن است مقدار count را دوباره بررسی کند. (باز هم، انتخاب یک نخ *دلخواه* مطابق توصیف جاوا می‌باشد؛ اکثر ماشین‌های مجازی جاوا در عمل، نخ‌های مجموعه‌ی منتظر را بر اساس یک سیاست FIFO مرتب می‌کنند.)

در ادامه، ما متدهای ()wait و ()notify را با تعابیر متدهای نشان داده شده در شکل ۷-۱۱ بیان می‌کنیم. فرض ما این است که بافر پر است و قفل شئ در دسترس می‌باشد.

- تولیدکننده متد ()insert را فراخوانی می‌کند، می‌بیند که قفل در دسترس است، و وارد متد می‌شود. تولیدکننده همین که در متد تعیین کرد که بافر پر است، ()wait را فراخوانی می‌کند. فراخوانی ()wait قفل شئ را رها می‌سازد، حالت تولیدکننده را مسدود انتخاب می‌کند و تولیدکننده را در مجموعه‌ی منتظر شئ قرار می‌دهد.
- مصرف‌کننده نهایتاً متد ()remove را فراخوانی کرده و وارد آن می‌شود، چرا که قفل شئ، اکنون در دسترس می‌باشد. مصرف‌کننده یک آیتم را از بافر برداشته و ()notify را فراخوانی می‌کند. توجه داشته باشید که مصرف‌کننده هنوز هم مالک قفل شئ می‌باشد.
- فراخوانی ()notify تولیدکننده را از مجموعه‌ی منتظر شئ حذف می‌کند، تولیدکننده را به مجموعه‌ی ورودی منتقل کرده و حالت تولیدکننده را اجرایی انتخاب می‌کند.
- مصرف‌کننده از متد ()remove خارج می‌شود. خروج از این متد، قفل شئ را رها می‌سازد.
- تولیدکننده تلاش می‌کند تا قفل را دومرتبه کسب کند و موفق می‌شود. آن اجرایش را از فراخوانی ()wait از سر می‌گیرد. تولیدکننده حلقه‌ی while را تست کرده، تعیین

می‌کند که جایی در بافر وجود دارد، و با باقیمانده‌ی متد ()insert پیشروی می‌کند. اگر هیچ نخی در مجموعه‌ی انتظار شئ وجود نداشته باشد، فراخوانی ()notify نادیده گرفته می‌شود. موقعی که تولیدکننده از متد خارج می‌شود، قفل شئ را رها می‌سازد.

راهکارهای synchronized، ()wait، و ()notify از اول پیدایش، بخشی از جاوا بوده‌اند. با این وجود، اصلاحیه‌های بعدی API جاوا، راهکارهای قفل‌گذاری انعطاف‌پذیرتر و قوی‌تری معرفی کردند که برخی از آن‌ها را در بخش‌های بعدی بررسی می‌کنیم.

۷-۴-۲ قفل‌های چندمدخلی

شاید ساده‌ترین راهکار قفل‌گذاری موجود در دسترس در API، ReentrantLock باشد. ReentrantLock در بسیاری از جهات مثل دستور synchronized که در بخش ۷-۴-۱ مطرح شد عمل می‌کند: یک ReentrantLock در مالکیت یک نخ منفرد می‌باشد و برای فراهم‌ساختن انحصار متقابل به‌منظور دسترسی یک منبع مشترک استفاده می‌شود. با این وجود، ReentrantLock چندین امکان دیگر هم فراهم می‌کند، همچون انتخاب یک پارامتر *fairness* که ترجیح می‌دهد قفل را به نخ با طولانی‌ترین انتظار اعطا کند. (به خاطر بیاورید که توصیف JVM نشان نمی‌دهد که نخ‌های مجموعه‌ی انتظار یک قفل شئ به یک ترتیب خاصی مرتب می‌شوند.)

یک نخ، یک ReentrantLock را با احضار متد ()lock آن کسب می‌کند. اگر قفل در دسترس باشد—یا اگر نخ احضارکننده‌ی ()lock از قبل مالک آن باشد که به همین دلیل هم **تعبیر چندمدخلی** دارد—()lock مالکیت قفل نخ احضارکننده را گمارده و کنترل را بر می‌گرداند. اگر قفل در دسترس نباشد، نخ احضارکننده مسدود می‌گردد تا این که در نهایت به قفل گمارده شود موقعی که مالک آن ()unlock را احضار می‌کند. ReentrantLock رابط Lock را پیاده‌سازی می‌کند که به‌صورت زیر استفاده می‌شود:

```
Lock key = new ReentrantLock();
key.lock ();
try {
        /* critical section */
}
finally {
        key.unlock ();
}
```

مدل برنامه‌نویسی با استفاده از try و finally به کمی توضیح نیاز دارد. اگر قفل از طریق متد ()lock کسب شود، مهم است که قفل با تشابهی هم رها شود. با گذاشتن ()unlock در یک عبارت finally به این اطمینان می‌رسیم که به‌محض این که بخش بحرانی تکمیل شود یا یک استثناء در میان بلاک try پیش بیاید قفل رها می‌گردد. توجه داشته باشید که ما فراخوانی به ()lock را در میان عبارت try نمی‌گذاریم، چرا که ()lock هرگونه استثنای بررسی شده را پرتاب نمی‌کند. در نظر بگیرید که اگر ما ()lock را در میان عبارت try بگذاریم چه اتفاقی می‌افتد و یک استثنای بررسی

نشده در زمان احضار ()lock رخ می‌دهد (همچون OutOfMemoryError): عبارت finally، فراخوانی ()unlock را راه می‌اندازد که در ادامه، استثنای بررسی نشده‌ی IllegalMonitorStateException را پرتاب می‌کند، چرا که قفل هرگز کسب نشده است. این استثنای IllegalMonitorStateException جایگزین استثنای بررسی نشده‌ای که موقع احضار ()lock اتفاق می‌افتد می‌شود که با این کار، دلیل این که چرا برنامه در آغاز با خطا مواجه می‌شود، مبهم است.

در حالی که ReentrantLock انحصار متقابل را فراهم می‌کند، اگر نخ‌های متعددی داده‌های مشترکی را فقط بخوانند، اما ننویسند، می‌تواند یک راهبرد بسیار محافظه‌کارانه باشد. (این سناریو را در بخش ۷-۱-۲ بیان کردیم.) برای لحاظ این نیاز، API جاوا یک ReentrantReadWriteLock نیز فراهم می‌کند، قفلی که به خوانندگان همروند متعدد اجازه می‌دهد، اما تنها یک نویسنده مجاز است.

۷-۴-۳ سمافور

API جاوا یک سمافور شمارشی نیز فراهم می‌کند، چنان‌چه در بخش ۶-۶ بیان شد. سازنده‌ی سمافور به‌صورت زیر است:

```
Semaphore(int value);
```

که value مقدار آغازین سمافور را مشخص می‌کند (مقدار منفی مجاز است). متد ()acquire در صورتی که نخ کسب‌کننده با وقفه مواجه شود یک استثنای InterruptedExecption پرتاب می‌کند. مثال زیر، استفاده از یک سمافور را برای انحصار متقابل نمایش می‌دهد:

```
Semaphore sem = new Semaphore(1);

try {
        sem.acquire();
        /* critical section */
}
catch (InterruptedException ie) { }
finally {
        sem.release();
}
```

توجه کنید که ما فراخوانی ()realease را در عبارت finally می‌گذاریم تا مطمئن شویم که سمافور رها می‌گردد.

۷-۴-۴ متغیرهای شرط

امکان مفید آخری که ما از API جاوا پوشش می‌دهیم متغیر شرط است. درست مثل شباهت ReentrantLock به دستور synchronized جاوا، متغیرهای شرط، کارکردی شبیه به متدهای ()wait و ()notify فراهم می‌کنند. لذا، برای تدارک انحصار متقابل، یک متغیر شرط بایستی با یک قفل چندمدخلی مرتبط شود.

ما یک متغیر شرط را نخست با ایجاد یک ReentrantLock و احضار متد ()newCondition آن ایجاد می‌کنیم که یک شئ Condition معرف متغیر شرط برای ReentrantLock مرتبط بر می‌گرداند. این موضوع با دستورات زیر نمایش داده می‌شود:

```
Lock key = new ReentrantLock();
Condition condVar = key.newCondition();
```

به محض این که متغیر شرط بدست آمد می‌توانیم متدهای ()await و ()signal آن را احضار کنیم که به همان شیوه‌ی فرمان‌های ()wait و ()signal که در بخش ۶-۷ مطرح شدند کار می‌کنند.

به‌خاطر بیاورید که در مانیتورها آن‌گونه که در بخش ۶-۷ بیان شد، عملیات ()wait و ()signal می‌توانند به متغیرهای شرط *با نام* اِعمال شوند که به یک نخ اجازه می‌دهد تا منتظر یک شرط خاصی بماند یا موقعی که یک شرط خاصی برآورده می‌شود اطلاع‌رسانی گردد. در سطح زبان، جاوا پشتیبانی برای متغیرهای شرط بی‌نام ندارد. هر مانیتور جاوا درست با یک متغیر شرط بی‌نام مرتبط شده و عملیات ()wait و ()signal مطرح شده در بخش ۷-۴-۱ فقط به این تک‌متغیر شرط اِعمال می‌شوند. موقعی که یک نخ جاوا از طریق ()notify بیدار می‌گردد، اطلاعاتی در رابطه با این که چرا بیدار شده دریافت نمی‌کند؛ برعهده‌ی نخی است که دومرتبه فعال شده است تا خودش بررسی کند که آیا شرطی که منتظر آن بوده است برآورده می‌شود یا خیر. متغیرهای شرط این مسئله را با اطلاع‌رسانی به یک نخ خاص حل می‌کنند.

ما این مطلب را با مثال زیر روشن می‌کنیم: فرض کنید که ما پنج از شماره ۰ تا ۴ نخ داریم و متغیر مشترک turn که نشان می‌دهد نوبت با کدام نخ است. موقعی که نخی تمایل به انجام کار دارد، متد ()doWork را در شکل ۷-۱۳ با ارسال شماره‌ی نخ خودش فراخوانی می‌کند. تنها نخی که مقدار threadNumber آن با مقدار turn تطبیق می‌نماید، می‌تواند پیشروی داشته باشد؛ سایر نخ‌ها بایستی منتظر نوبتشان باشند.

بایستی یک ReentrantLock و پنج متغیر شرط (معرف شروطی که نخ‌ها منتظرند) ایجاد کنیم تا نخی را که نوبت بعدی با آن است را علامت بدهند. این موضوع به‌صورت زیر نشان داده می‌شود:

```
Lock lock = new ReentrantLock();
Condition[] condVars = new Condition[5];

for (int i = 0; i < 5; i++)
    condVars[i] = lock.newCondition();
```

موقعی که نخی وارد ()doWork می‌شود، متد ()await را بر روی متغیر شرط مرتبطش احضار می‌کند در صورتی که threadNumber آن با turn برابر نباشد، تنها زمانی از سر گرفته می‌شود که توسط نخ دیگری سیگنال داده شود. بعد از این که یک نخ کارش را تکمیل کرد، متغیر شرط مرتبط با نخی را که در نوبت بعدی است سیگنال می‌دهد.

توجه به این نکته مهم است که لازم نیست ()doWork به‌صورت synchronized اعلان شود، چرا که ReentrantLock انحصار متقابل را فراهم می‌کند. موقعی که یک نخ، ()await را بر روی

متغیر شرط احضار می‌کند، ReentrantLock مرتبط را رها می‌سازد که با این کار، نخ دیگر اجازه می‌یابد تا قفل انحصار متقابل را کسب کند. با تشابهی، موقعی که ()signal احضار می‌شود، تنها متغیر شرط سیگنال داده می‌شود؛ قفل با احضار ()unlock رها می‌گردد.

```
/* threadNumber is the thread that wishes to do some work */
public void doWork(int threadNumber)
{
        lock.lock();
        try {
                /**
                * If it's not my turn, then wait
                * until I'm signaled.
                */
                if (threadNumber != turn)
                        condVars[threadNumber].await();
                /**
                * Do some work for awhile ...
                */
                /**
                * Now signal to the next thread.
                */
                turn = (turn + 1) % 5;
                condVars[turn].signal();
        }
        catch (InterruptedException ie) { }
        finally {
                lock.unlock();
        }
}
```

شکل ۷-۱۳ مثال استفاده از متغیرهای شرط جاوا.

۷-۵ رویکردهای جایگزین

با پیدایش سیستم‌های چندهسته‌ای، فشار مضاعفی بر توسعه‌ی کاربردهای چندنخی است تا از مزیت هسته‌های پردازشی متعدد بهره ببرند. با این وجود، برنامه‌های همروند، خطر رو به افزایش شرایط مسابقه و خطرات عاقبت به‌خیری، همچون بن‌بست را پیش می‌کشند. در سابق، تکنیک‌هایی از قبیل قفل‌های انحصار متقابل، سمافورها و مانیتورها برای رفع این نگرانی‌ها استفاده می‌شد، اما با افزایش تعداد هسته‌های پردازشی، طراحی کاربردهای چندنخی که عاری از شرایط مسابقه و بن‌بست باشند به صورت فزاینده دشوارتر می‌شود. در این بخش، امکانات گوناگونی را مرور می‌کنیم که در هر دوی زبان‌های برنامه‌نویسی و سخت‌افزار که طراحی کاربردهای همروند نخ ایمن را پشتیبانی می‌کنند، فراهم شده است.